An Analysis of Masked language model

Masked language model

Masked language model -- Part 1

Embedding This section describes the embedding used by BERTBASE. The other one, BERTLARGE, is similar, just larger. The tokenizer of BERT is WordPiece, which is a sub-word strategy like byte-pair encoding. Its vocabulary size is 30,000, and any token not appearing in its vocabulary is replaced by [UNK] ("unknown").

Masked language model -- Part 2

with probability 80%, the chosen token is masked, resulting in "my1 dog2 is3 [MASK]4"; with probability 10%, the chosen token is replaced by a uniformly sampled random token, such as "happy", resulting in "my1 dog2 is3 happy4"; with probability 10%, nothing is done, resulting in "my1 dog2 is3 cute4". After processing the input text, the model's 4th output vector is passed to its decoder layer, which outputs a probability distribution over its 30,000-dimensional vocabulary space.

Masked language model -- Part 3

Masked language modeling (MLM): In this task, BERT ingests a sequence of words, where one word may be randomly changed ("masked"), and BERT tries to predict the original words that had been changed. For example, in the sentence "The cat sat on the [MASK]," BERT would need to predict "mat." This helps BERT learn bidirectional context, meaning it understands the relationships between words not just from left to right or right to left but from both directions at the same time. Next sentence prediction (NSP): In this task, BERT is trained to predict whether one sentence logically follows another. For example, given two sentences, "The cat sat on the mat" and "It was a sunny day", BERT has to decide if the second sentence is a valid continuation of the first one. This helps BERT understand relationships between sentences, which is important for tasks like question answering or document classification.

Masked language model -- Part 4

Architectural family The encoder stack of BERT has 2 free parameters: L {\displaystyle L} , the number of layers, and H {\displaystyle H} , the hidden size. There are always H / 64 {\displaystyle H/64} self-attention heads, and the feed-forward/filter size is always 4 H {\displaystyle 4H} . By varying these two numbers, one obtains an entire family of BERT models. For BERT:

Masked language model -- Part 5

Tokenizer: This module converts a piece of English text into a sequence of integers ("tokens"). Embedding: This module converts the sequence of tokens into an array of real-valued vectors representing the tokens. It represents the conversion of discrete token types into a lower-dimensional Euclidean space. Encoder: a stack of Transformer blocks with self-attention, but without causal masking. Task head: This module converts the final representation vectors into one-shot encoded tokens again by producing a predicted probability distribution over the token types. It can be viewed as a simple decoder, decoding the latent representation into token types, or as an "un-embedding layer". The task head is necessary for pre-training, but it is often unnecessary for so-called "downstream tasks," such as question answering or sentiment classification. Instead, one removes the task head and replaces it with a newly initialized module suited for the task, and finetune the new module. The latent vector representation of the model is directly fed into this new module, allowing for sample-efficient transfer learning.

Masked language model -- Part 6

the feed-forward size and filter size are synonymous. Both of them denote the number of dimensions in the middle layer of the feed-forward network. the hidden size and embedding size are synonymous. Both of them denote the number of real numbers used to represent a token. The notation for encoder stack is written as L/H. For example, BERTBASE is written as 12L/768H, BERTLARGE as 24L/1024H, and BERTTINY as 2L/128H.

Masked language model -- Part 7

Interpretation Language models like ELMo, GPT-2, and BERT, spawned the study of "BERTology", which attempts to interpret what is learned by these models. Their performance on these natural language understanding tasks are not yet well understood. Several research publications in 2018 and 2019 focused on investigating the relationship behind BERT's output as a result of carefully chosen input sequences, analysis of internal vector representations through probing classifiers, and the relationships represented by attention weights. The high performance of the BERT model could also be attributed to the fact that it is bidirectionally trained. This means that BERT, based on the Transformer model architecture, applies its self-attention mechanism to learn information from a text from the left and right side during training, and consequently gains a deep understanding of the context. For example, the word fine can have two different meanings depending on the context (I feel fine today, She has fine blond hair). BERT considers the words surrounding the target word fine from the left and right side. However it comes at a cost: due to encoder-only architecture lacking a decoder, BERT can't be prompted and can't generate

text, while bidirectional models in general do not work effectively without the right side, thus being difficult to prompt. As an illustrative example, if one wishes to use BERT to continue a sentence fragment "Today, I went to", then naively one would mask out all the tokens as "Today, I went to [MASK] [MASK] [MASK] ... [MASK] ." where the number of [MASK] is the length of the sentence one wishes to extend to. However, this constitutes a dataset shift, as during training, BERT has never seen sentences with that many tokens masked out. Consequently, its performance degrades. More sophisticated techniques allow text generation, but at a high computational cost.

Masked language model -- Part 8

Variants The BERT models were influential and inspired many variants. RoBERTa (2019) was an engineering improvement. It preserves BERT's architecture (slightly larger, at 355M parameters), but improves its training, changing key hyperparameters, removing the next-sentence prediction task, and using much larger mini-batch sizes. XLM-RoBERTa (2019) was a multilingual RoBERTa model. It was one of the first works on multilingual language modeling at scale. DistilBERT (2019) distills BERTBASE to a model with just 60% of its parameters (66M), while preserving 95% of its benchmark scores. Similarly, TinyBERT (2019) is a distilled model with just 28% of its parameters. ALBERT (2019) used shared-parameter across layers, and experimented with independently varying the hidden size and the word-embedding layer's output size as two hyperparameters. They also replaced the next sentence prediction task with the sentence-order prediction (SOP) task, where the model must distinguish the correct order of two consecutive text segments from their reversed order. ELECTRA (2020) applied the idea of generative adversarial networks to the MLM task. Instead of masking out tokens, a small language model generates random plausible substitutions, and a larger network identify these replaced tokens. The small model aims to fool the large model. DeBERTa (2020) is a significant architectural variant, with disentangled attention. Its key idea is to treat the positional and token encodings separately throughout the attention mechanism. Instead of combining the positional encoding (x p o s i t i o n {\displaystyle x_{\mathrm {position} }}) and token encoding (x t o k e n {\displaystyle x_{\mathrm {token} }}) into a single input vector (x i n p u t = x p o s i t i o n + x t o k e n {\displaystyle x_{\mathrm {input} }=x_{\mathrm {position} }+x_{\mathrm {token} }}), DeBERTa keeps them separate as a tuple: (x p o s i t i o n , x t o k e n) {\displaystyle (x_{\mathrm {position} },x_{\mathrm {token} })} . Then, at each self-attention layer, DeBERTa computes three distinct attention matrices, rather than the single attention matrix used in BERT: